# Operation Lockout

## A Multi-SIEM Framework for Detecting, Correlating, and Investigating Brute-Force Attacks Against Active Directory

**Name:** Aigbokhaode Hope Imomoh **Date:** 9-10-2026 **Program/Course:** SOC Analysis Home Lab

**Lab Duration:** 3 hours 45 minutes **Difficulty:** Intermediate

*Work spanned three sessions: initial build and attack on 2026-10-05, Splunk dashboard construction and re-testing on 2026-10-07, and search validation on 2026-10-14.*

## Executive Summary

This project simulates, detects, and investigates a credential brute-force attack against an Active Directory environment using a dual-SIEM architecture (Wazuh and Splunk), portable Sigma detection rules, and MITRE ATT&CK mapping. Using CrackMapExec, a dictionary attack against a domain account (HR-STAFF1) succeeded in recovering a weak password, generating 22,069 security alerts over 24 hours, of which 21,898 were level-12-or-above. A custom two-stage Wazuh correlation rule — a single failed-logon detector escalating to a brute-force alert once 5 failures occur within 2 minutes — and an independently built Splunk SPL query and dashboard both caught the attack and flagged the compromised account in real time. The result is a complete, evidence-backed detection-to-investigation workflow: log collection, correlation-rule engineering, cross-platform rule portability via Sigma, and root-cause analysis, mapped throughout to MITRE ATT&CK technique T1110 (Brute Force).

## Objective

To develop practical SOC Analyst skills in security log collection, brute-force detection, SIEM investigation, threat analysis, and incident prevention using Wazuh and Splunk.

## Lab Environment and Tools

This lab was implemented in a controlled virtual cybersecurity lab using Oracle VirtualBox to host the isolated systems required for attack simulation, security monitoring, and incident investigation. The environment integrates Windows Server 2019 with Active Directory for centralized authentication and user management, Windows endpoint systems for generating security events, and Ubuntu Linux for hosting Wazuh for security monitoring and alert analysis. Splunk is used for searching, event correlation, and dashboard development. Additional tools include Windows Event Viewer for native Windows log inspection, Sysmon for enhanced endpoint telemetry, and Sigma for portable detection rule development.

**Host machine:** Dell Latitude 7290

**Network adapters (all VMs):** Adapter 1 = NAT, Adapter 2 = Host-only

| VM | Role | OS | IP address |
|---|---|---|---|
| VM1 | Wazuh manager/indexer and Splunk indexer | Ubuntu | 192.168.56.102 / .101 |
| VM2 | Target system hosting AD users (HR-STAFF1) | Windows 10 | 192.168.56.116 |
| VM3 | Domain controller hosting Active Directory | Windows Server 2019 | 192.168.56.115 |
| VM4 | Attacker machine: Sigma, CrackMapExec, wordlists | Kali Linux | 192.168.56.103 |

## Tools and Technologies

- SIEM platforms: Wazuh and Splunk
- Endpoint telemetry: Sysmon
- Log investigation: Windows Event Viewer
- Detection engineering: Sigma rules and custom Wazuh rules
- Attack framework: MITRE ATT&CK

## Lab Configuration

The environment consists of a domain controller, a Windows client, a Linux-based monitoring system server, and a controlled attack-simulation system. Authentication events are generated within the lab, collected by the monitoring platform, and analyzed to identify brute-force patterns and evaluate detection effectiveness. All testing is performed on authorized lab systems using designated test accounts and controlled network access.

# Prerequisites and Base Infrastructure Setup

Everything below is written so a reader can rebuild the environment from nothing and reproduce the same detections. Download and have ready before starting:

| Component | Source |
|---|---|
| Oracle VirtualBox (+ Extension Pack) | virtualbox.org |
| Windows Server 2019 evaluation ISO | microsoft.com/evalcenter |
| Windows 10 ISO | microsoft.com/software-download |
| Ubuntu Server 22.04 LTS ISO | ubuntu.com/download/server |
| Kali Linux ISO | kali.org/get-kali |
| SecLists wordlists | github.com/danielmiessler/SecLists |
| Wazuh 4.x | packages.wazuh.com |
| Splunk Enterprise / Universal Forwarder | splunk.com/download |
| Sysmon + a community config (e.g. SwiftOnSecurity) | learn.microsoft.com/sysinternals/downloads/sysmon |

## 0. Base Infrastructure Setup

**0.1.** Install Oracle VirtualBox and the Extension Pack on the host machine.

**0.2.** Create four VMs, each with two adapters — Adapter 1: NAT, Adapter 2: Host-only (VirtualBox's default host-only network, 192.168.56.0/24):

| VM | OS | Suggested specs | Role |
|---|---|---|---|
| VM1 | Ubuntu Server 22.04 LTS | 4 vCPU / 8 GB RAM / 60 GB disk | Wazuh manager + indexer, Splunk |
| VM2 | Windows 10 | 2 vCPU / 4 GB RAM / 60 GB disk | Domain-joined endpoint (DESKTOP-T6T1N2V) |
| VM3 | Windows Server 2019 | 2 vCPU / 4 GB RAM / 80 GB disk | Domain controller |
| VM4 | Kali Linux | 2 vCPU / 4 GB RAM / 40 GB disk | Attacker box |

**0.3.** On VM3, install the AD DS role and promote it to a new forest:

```
Install-WindowsFeature AD-Domain-Services -IncludeManagementTools
Install-ADDSForest -DomainName "HopeLabs.Local" -DomainNetbiosName "HOPELABS" -InstallDNS
```

**0.4.** In Active Directory Users and Computers (or via New-ADUser), create an OU and the test accounts, including the target account used later in the attack:

```
New-ADUser -Name "HR-STAFF1" -SamAccountName "HR-STAFF1" \
  -AccountPassword (Read-Host -AsSecureString "Password") -Enabled $true
```

**0.5.** Join VM2 (Windows 10) to HopeLabs.Local and confirm with (Get-WmiObject Win32_ComputerSystem).Domain.

**0.6.** Install Sysmon on VM2 for enhanced endpoint telemetry:

```
Sysmon64.exe -accepteula -i sysmonconfig.xml
```

**0.7.** Install the Wazuh manager and indexer on VM1 using the official all-in-one installer:

```
curl -sO https://packages.wazuh.com/4.x/wazuh-install.sh
sudo bash wazuh-install.sh -a
```

**0.8.** Install the Wazuh agent on VM2 and VM3, pointing each at the manager's host-only IP (192.168.56.102), then enroll and start the service (exact commands in Methodology section C).

**0.9.** Install Splunk Enterprise on VM1:

```
wget -O splunk.tgz "<Splunk Enterprise .tgz download URL>"
sudo tar xvzf splunk.tgz -C /opt
sudo /opt/splunk/bin/splunk start --accept-license
```

**0.10.** Install the Splunk Universal Forwarder on VM2, point it at the Ubuntu indexer (192.168.56.102:9997), and configure it to monitor the Windows Security event log (WinEventLog://Security) so EventCode=4625 events reach Splunk alongside Wazuh.

With the four VMs built, domain established, and both SIEMs installed and talking to their agents/forwarders, the environment matches the one described above and the numbered procedure in Methodology can be followed exactly as written.

## Methodology / Procedure

### A. Preparing the Attacker (Kali Linux)

**1.** Using SecLists, built a usernames.txt (1,000+ entries) and passwords.txt (1,000+ entries) for brute-forcing.

**2.** Installed CrackMapExec (sudo apt install crackmapexec -y) and confirmed the version.

**3.** Confirmed connectivity and protocol access to the Windows 10 VM before the attack run.

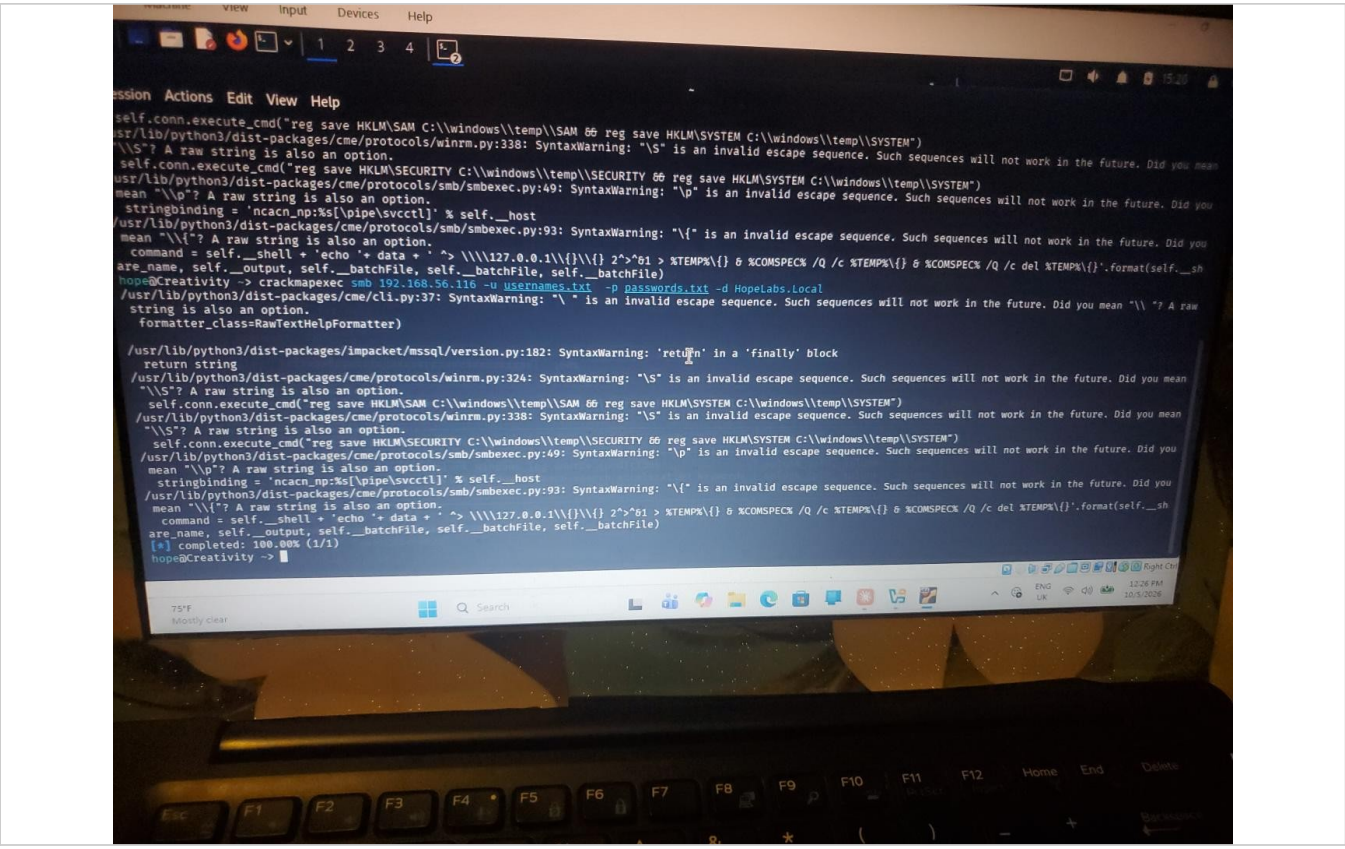



*Figure 1 — CrackMapExec pre-attack connectivity check against the Windows 10 VM (192.168.56.116) completing successfully.*

### B. Verifying the Ubuntu SIEM Host (VM1)

**4.** Checked the Ubuntu IP (ip addr → 192.168.56.102) and OS version (lsb_release -a).

**5.** Checked Wazuh services: sudo systemctl status wazuh-manager / wazuh-indexer.

**6.** Checked Splunk: sudo systemctl status splunk; confirmed install at /opt/splunk/bin/splunk.

**7.** Verified network connectivity to the Windows Server and Windows client (ping -c 4).

**8.** Started Wazuh and Splunk, and confirmed both reported active status.

**9.** Opened Splunk (http://192.168.56.102:8000) and the Wazuh dashboard (https://192.168.56.102:443) from Firefox.

## C. Configuring the Windows 10 Agent (VM2)

**10.** Opened PowerShell as Administrator and checked the Windows IP (ipconfig).

**11.** Tested connectivity to Wazuh (ping 192.168.56.102) and verified the agent communication port (1514) and enrollment port (1515) with Test-NetConnection — both returned Test succeeded: True.

**12.** Checked and started the Wazuh agent service: Get-Service WazuhSvc / Start-Service WazuhSvc.

## D. Installing Sigma on Kali

**13.** Created a project directory (mkdir -p Sigma_project/rules) and installed prerequisites (git, python3, python3-venv, pipx).

**14.** Installed the Sigma CLI in an isolated environment with pipx and verified with sigma --version.

**15.** Installed the Splunk, Wazuh, OpenSearch, and Sysmon Sigma plugins and verified with sigma plugin list.

**16.** Optionally cloned the official SigmaHQ rule repository (~/sigma-official) for reference.

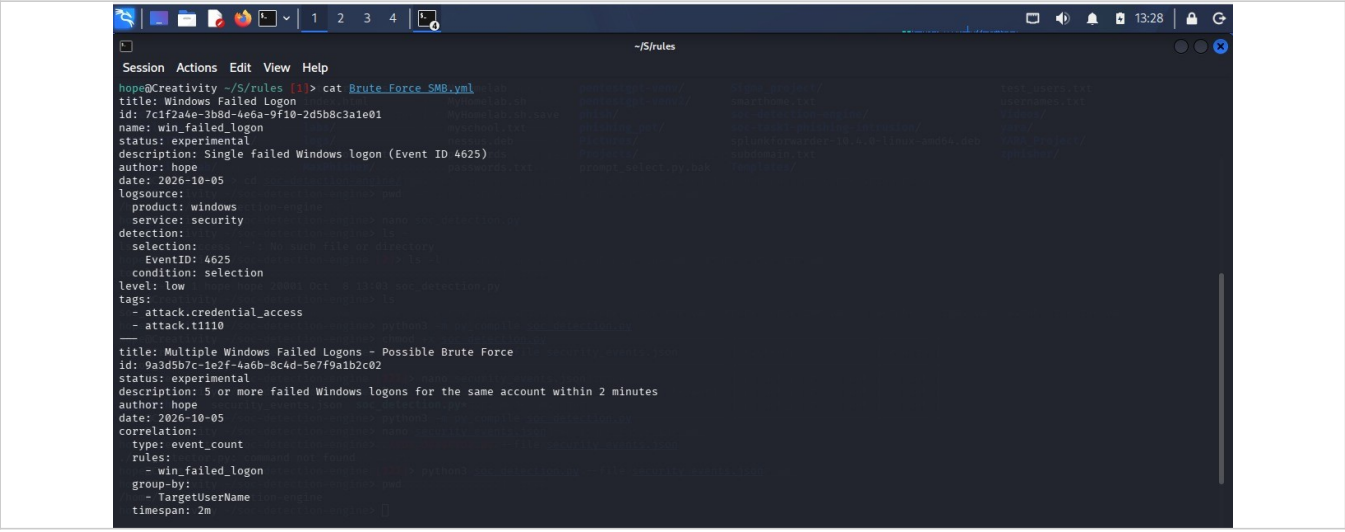



*Figure 2 — Sigma project directory, the official SigmaHQ repository cloned to ~/sigma-official, and confirming the custom rule file by exact filename.*

## E. Running the Brute-Force Attack

**17.** Launched the brute-force attack against the target AD user (HR-STAFF1) with CrackMapExec:

```
crackmapexec smb 192.168.56.116 -u usernames.txt -p passwords.txt -d HopeLabs.Local
```

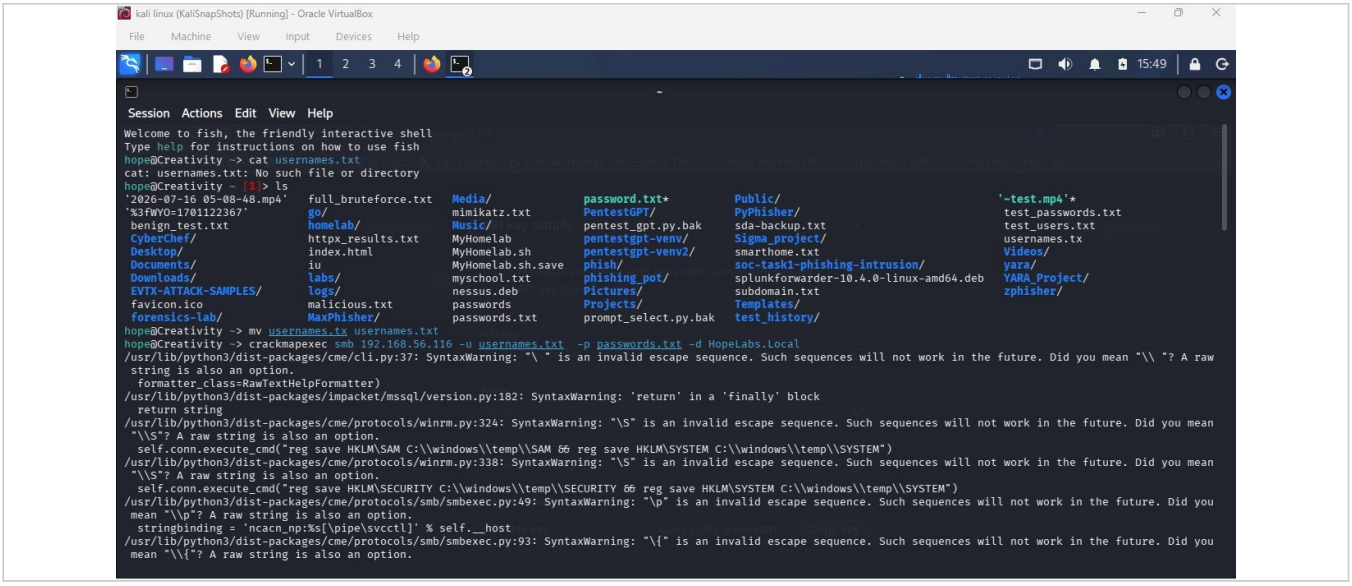



*Figure 3 — The CrackMapExec brute-force run launched from the Kali Creativity VM against the HopeLabs.Local domain.*

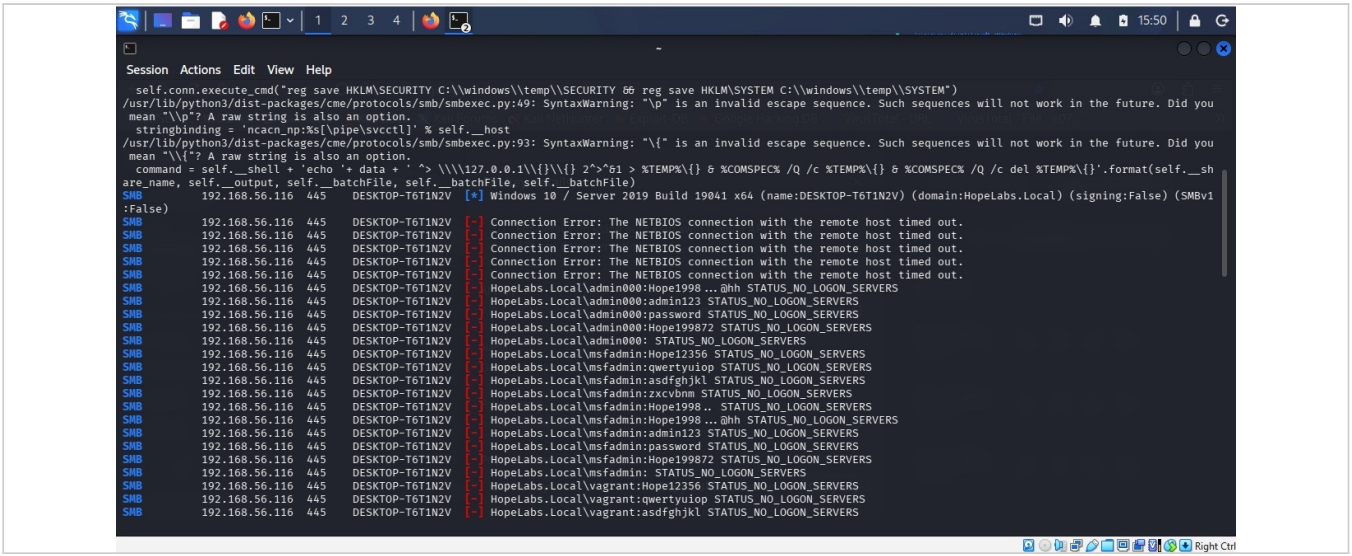



*Figure 4 — CrackMapExec fingerprinting the target (Windows 10/Server 2019 Build 19041, SMB signing disabled) and cycling through generic accounts, all rejected with STATUS_NO_LOGON_SERVERS.*

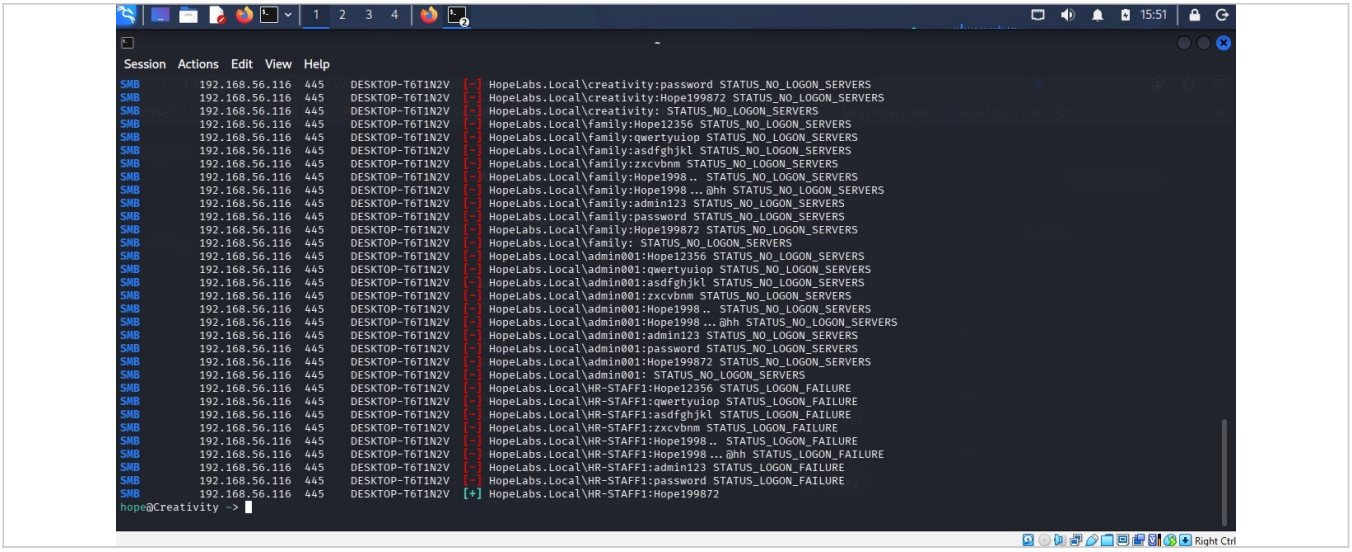



*Figure 5 — The attack reaching the real domain account HR-STAFF1 and succeeding: [+] HopeLabs.Local\HR-STAFF1:Hope199872.*

## F. Writing the Sigma Detection Rule

**18.** Authored two Sigma rules in Brute_Force_SMB.yml: a single-event detector (win_failed_logon, Event ID 4625) and a correlation rule (5+ failed logons for the same TargetUserName within a 2-minute timespan, escalated to level: high).

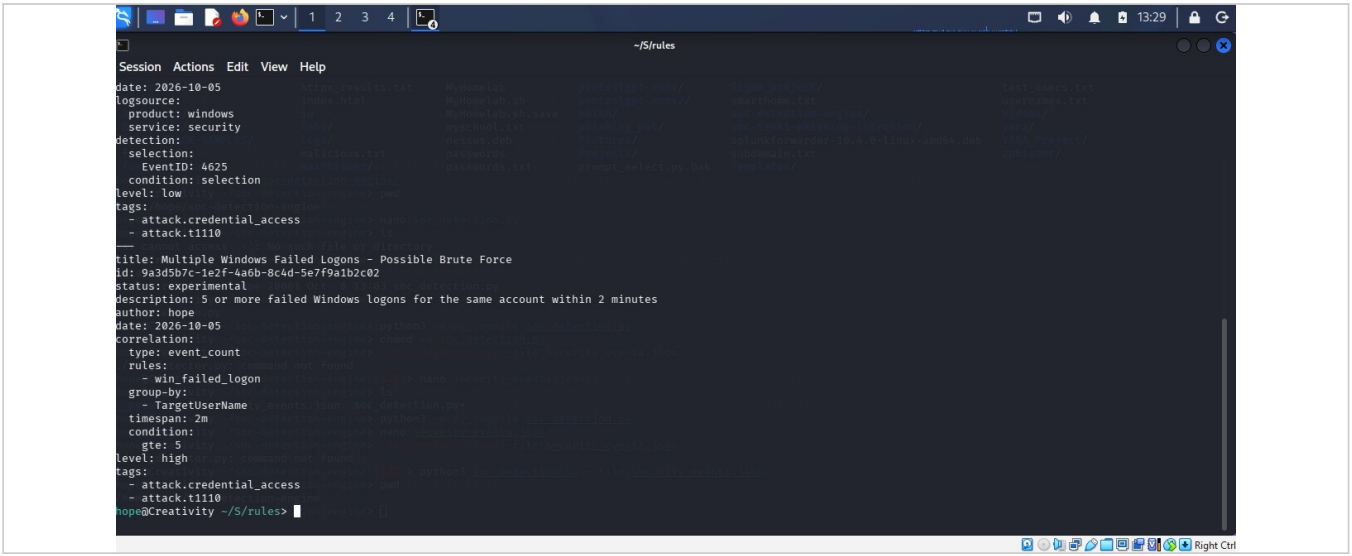



*Figure 6 — The deployed Brute_Force_SMB.yml displayed in full: the single-event rule and the event_count correlation rule grouped by TargetUserName.*

**19.** Checked the installed OpenSearch Lucene pipelines (sigma list pipelines opensearch_lucene) — none were registered by default, so a custom wazuh_pipeline.yml field-mapping pipeline was authored before conversion.

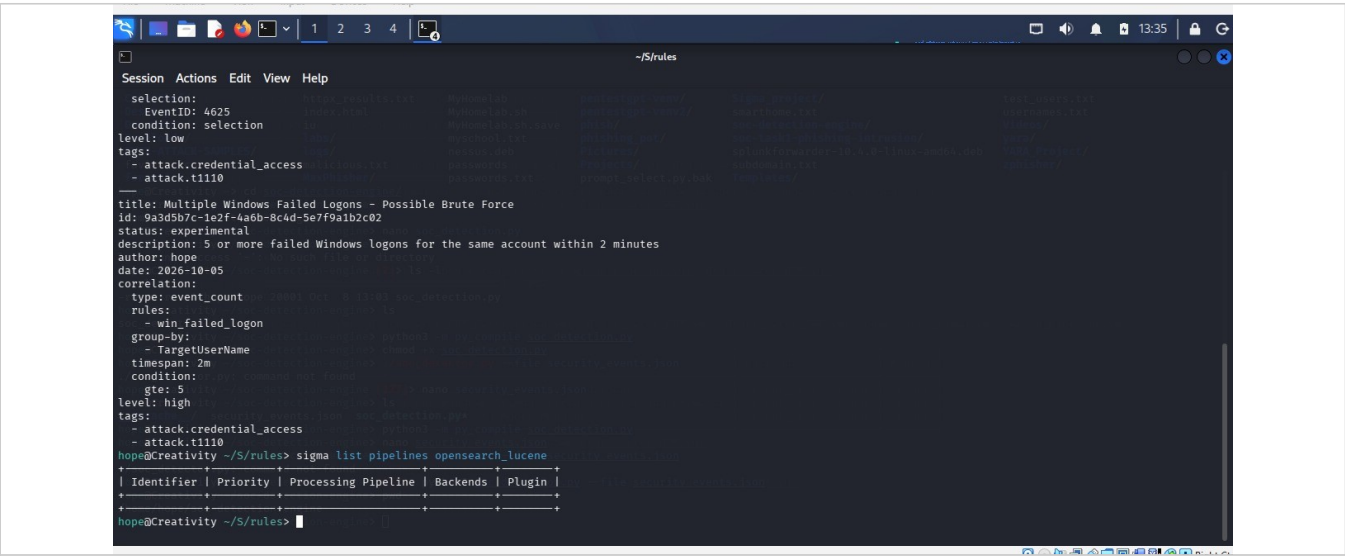



*Figure 7 — sigma list pipelines opensearch_lucene returning an empty table, confirming no backend pipeline ships by default.*

## G. Deploying the Wazuh and Splunk Detections

**20.** Built the Splunk SPL query grouping EventCode=4625 by source IP with a 5-attempt threshold.

**21.** Wrote and deployed the custom Wazuh rules in /var/ossec/etc/rules/local_rules.xml: rule 100100 (single failed Windows logon, level 10) and rule 100101 (5 matches of 100100 within 120 seconds, level 12, MITRE T1110, group authentication_failures/brute_force).

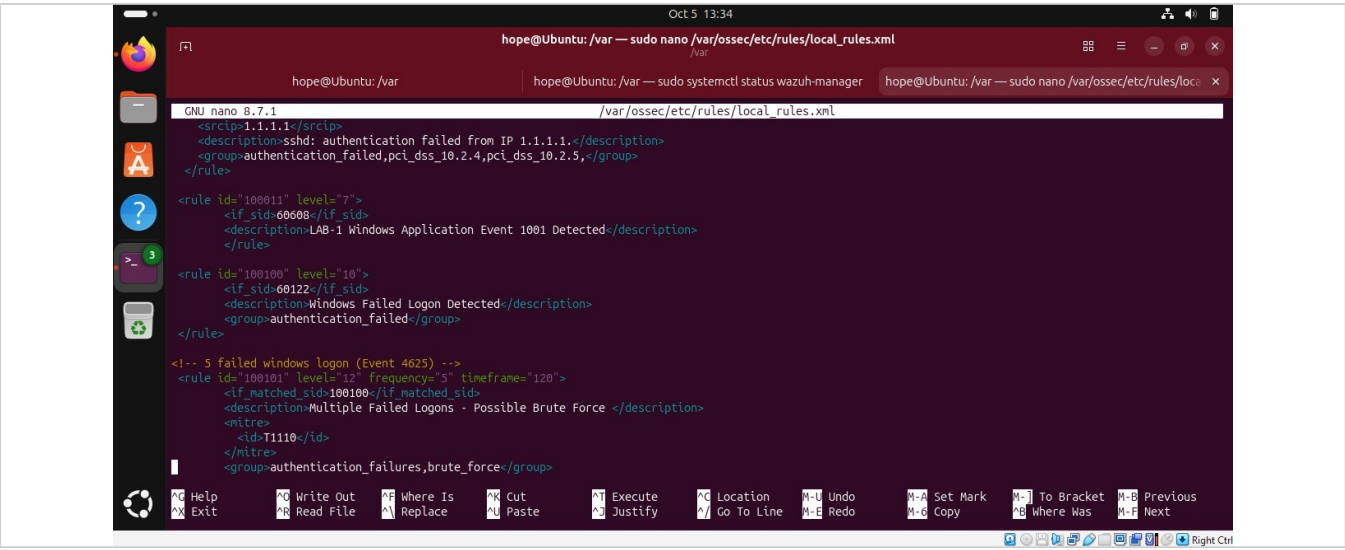



*Figure 8 — The deployed local_rules.xml open in nano, showing rules 100100 and 100101 exactly as they run on the Wazuh manager.*

**22.** Validated the new rule syntax with sudo /var/ossec/bin/wazuh-analysisd -t (successful output).

## H. Troubleshooting and Live Validation

**23.** Attempted to convert the correlation rule (the 5-in-120-seconds logic) to OpenSearch — the Lucene backend rejected it: "Backend does not support correlation rules." The single-event RDP rule converted cleanly instead, confirming the limitation was backend-specific, not a rule-authoring error.

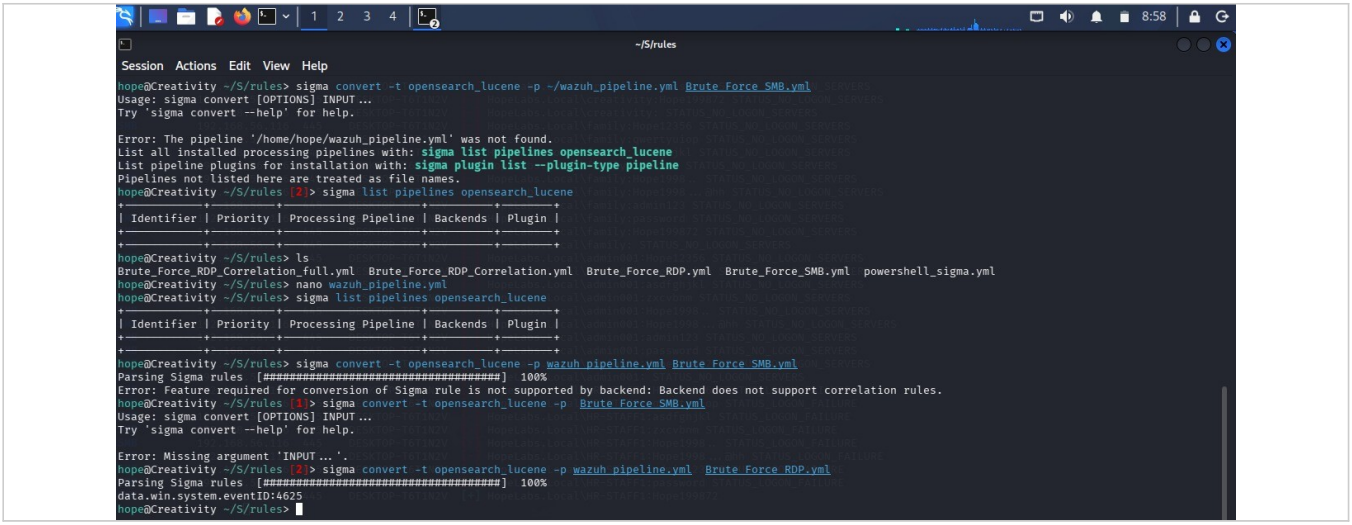



*Figure 9 — The failed correlation-rule conversion (top) alongside the successful single-event RDP rule conversion to data.win.system.eventID:4625 (bottom).*

**24.** Confirmed, from the Wazuh Threat Hunting dashboard and event log, that rules 100100/100101 fired as designed and that the attack ultimately succeeded against HR-STAFF1.

## Results / Observations

### Wazuh Detection — 24-Hour Overview

Over the 24 hours spanning the exercise (Oct 4 13:23 – Oct 5 13:23), the Wazuh Threat Hunting dashboard for agent DESKTOP-T6T1N2V recorded:

| Metric | Value |
|---|---|
| Total alerts | 22,069 |
| Level 12+ alerts | 21,898 |
| Authentication failures | 118 |
| Authentication successes | 2 |

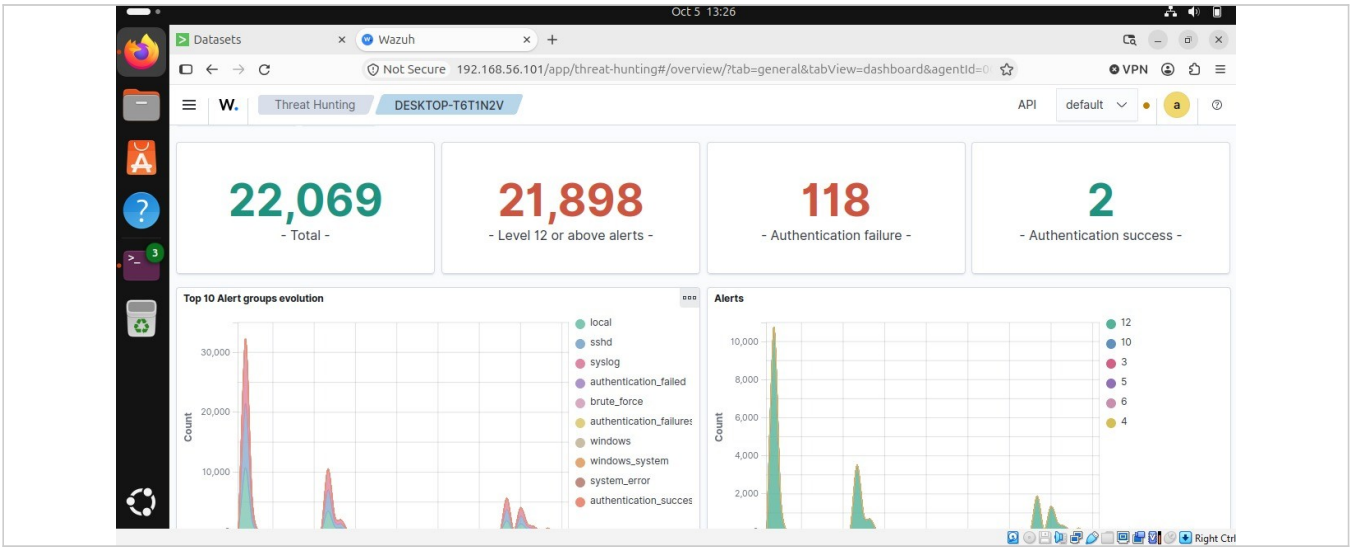



*Figure 10 — Wazuh dashboard summary statistics for agent DESKTOP-T6T1N2V over the 24-hour window.*

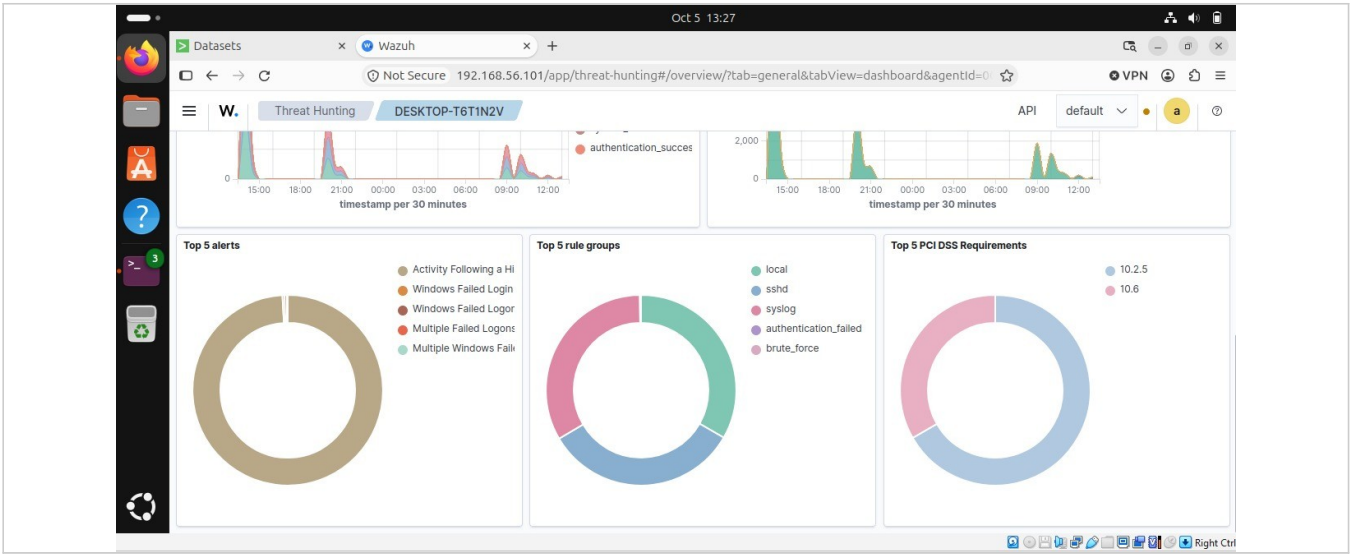



*Figure 11 — Top 5 alerts, top 5 rule groups (local, sshd, syslog, authentication_failed, brute_force) and top PCI DSS requirements (10.2.5, 10.6).*

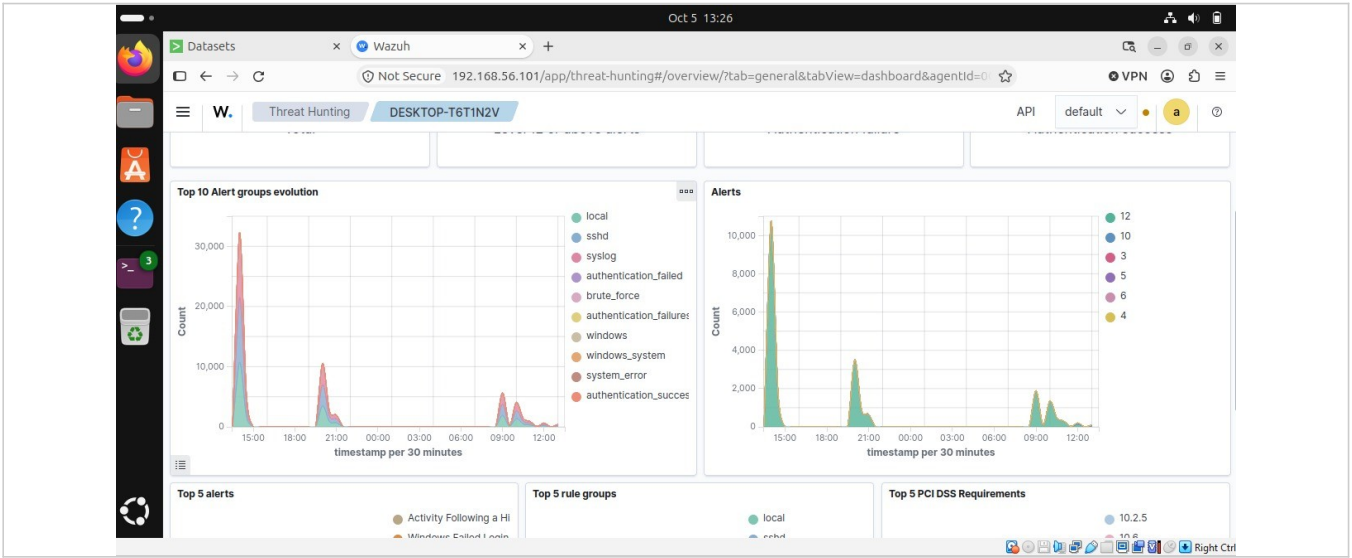
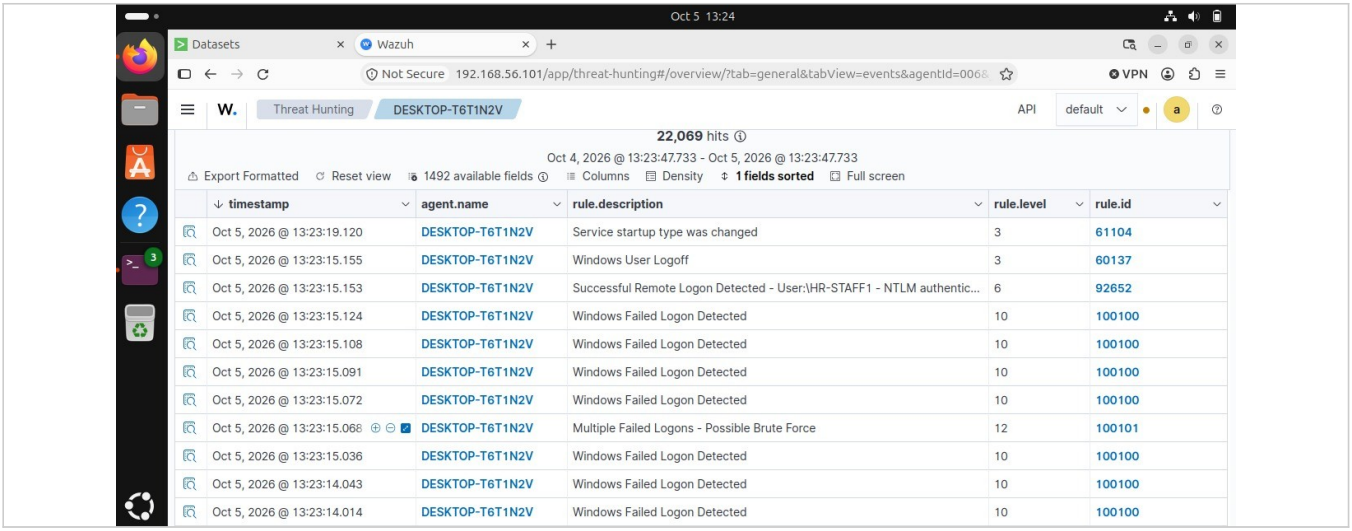



*Figure 12 — Top 10 alert-group evolution and alert-level timeline, both showing three matching activity bursts aligned with the brute-force runs.*

## Wazuh Detection — Event-Level Detail

Filtering the event log to the DESKTOP-T6T1N2V agent returned 667 pages (15 rows/page, roughly 10,000 matching events). The sequence is consistent end to end: repeated rule 100100 hits, rule 100101 escalating once the 5-in-120-second threshold is met, and a Successful Remote Logon (rule 92652) for HR-STAFF1 logged immediately after — the signature of a successful brute force, not a false positive.




| timestamp | agent.name | rule.description | rule.level | rule.id |
|---|---|---|---|---|
| Oct 5, 2026 @ 13:23:19.120 | DESKTOP-T6T1N2V | Service startup type was changed | 3 | 61104 |
| Oct 5, 2026 @ 13:23:15.155 | DESKTOP-T6T1N2V | Windows User Logoff | 3 | 60137 |
| Oct 5, 2026 @ 13:23:15.153 | DESKTOP-T6T1N2V | Successful Remote Logon Detected - User:\HR-STAFF1 - NTLM authentic... | 6 | 92652 |
| Oct 5, 2026 @ 13:23:15.124 | DESKTOP-T6T1N2V | Windows Failed Logon Detected | 10 | 100100 |
| Oct 5, 2026 @ 13:23:15.108 | DESKTOP-T6T1N2V | Windows Failed Logon Detected | 10 | 100100 |
| Oct 5, 2026 @ 13:23:15.091 | DESKTOP-T6T1N2V | Windows Failed Logon Detected | 10 | 100100 |
| Oct 5, 2026 @ 13:23:15.072 | DESKTOP-T6T1N2V | Windows Failed Logon Detected | 10 | 100100 |
| Oct 5, 2026 @ 13:23:15.068 | DESKTOP-T6T1N2V | Multiple Failed Logons - Possible Brute Force | 12 | 100101 |
| Oct 5, 2026 @ 13:23:15.036 | DESKTOP-T6T1N2V | Windows Failed Logon Detected | 10 | 100100 |
| Oct 5, 2026 @ 13:23:14.043 | DESKTOP-T6T1N2V | Windows Failed Logon Detected | 10 | 100100 |
| Oct 5, 2026 @ 13:23:14.014 | DESKTOP-T6T1N2V | Windows Failed Logon Detected | 10 | 100100 |

*Figure 13 — Event log showing rule 100100/100101 escalation and the Successful Remote Logon Detected event for HR-STAFF1 via NTLM.*

## Splunk Dashboard and Event-Level Evidence

A Splunk dashboard, SOC Brute Force Detection, was built with a "Failed Logon By Source IP" panel (EventCode=4625, last 24 hours). It shows failed logons overwhelmingly from 192.168.56.102 (the Wazuh/Splunk indexer host) with a smaller share from 127.0.0.1 (local loopback).

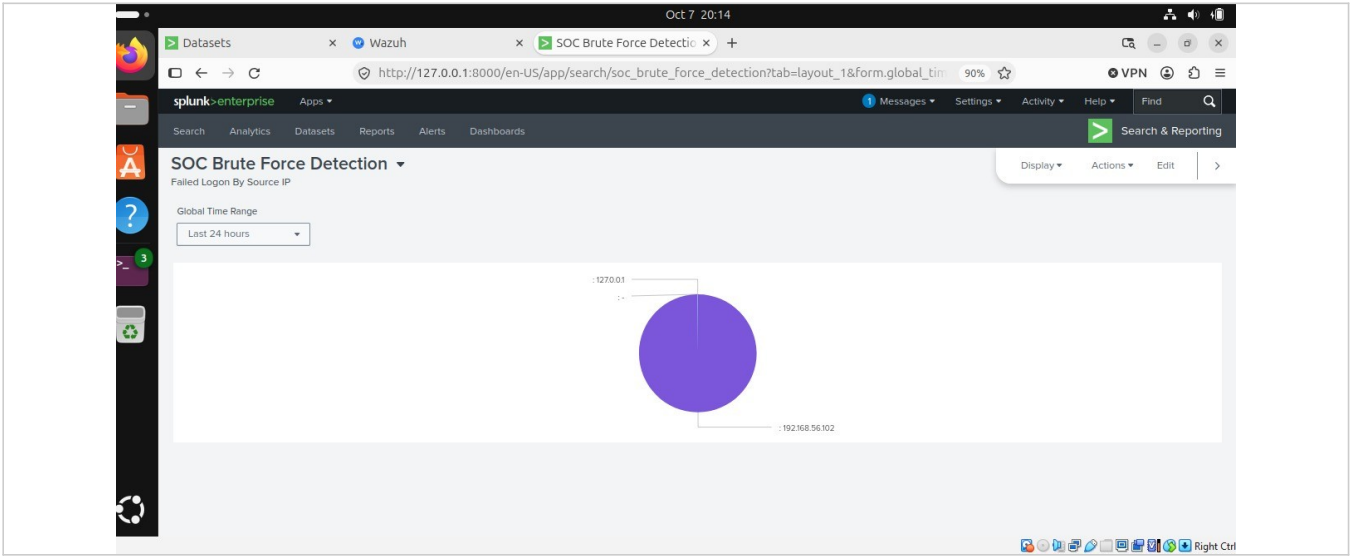



*Figure 14 — The SOC Brute Force Detection Splunk dashboard, Failed Logon By Source IP panel.*

Drilling into the raw EventCode=4625 events in Splunk surfaced the full Windows Security log detail behind the alert counts — failure reason, status/sub-status codes, workstation, and the targeted account (HR-STAFF1) — independently corroborating the Wazuh-side findings.

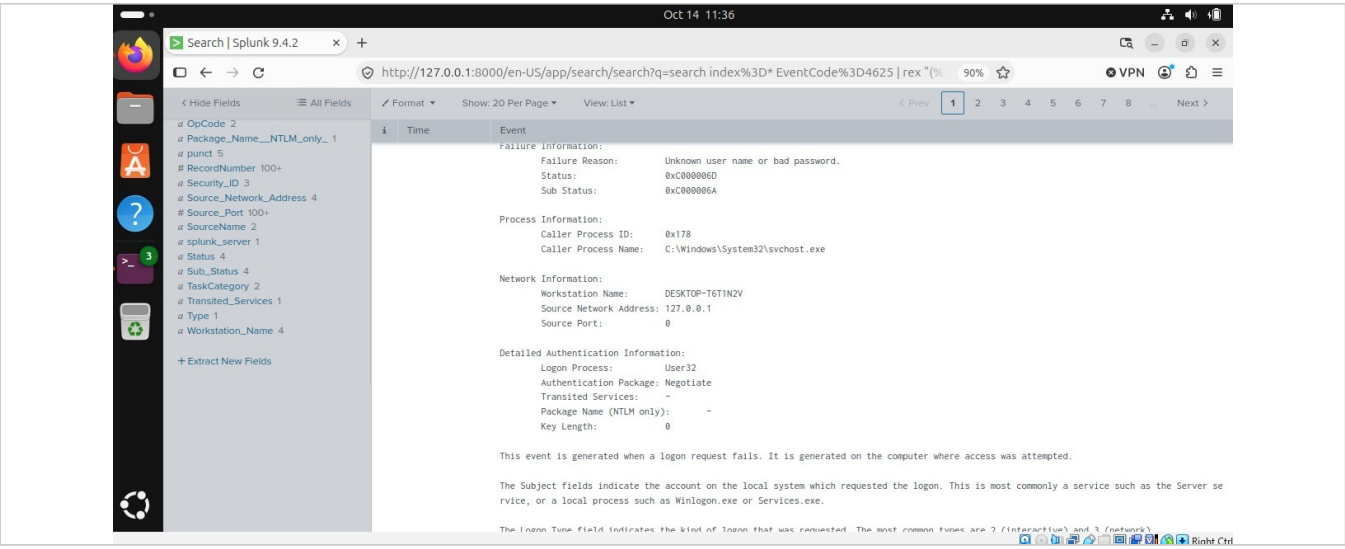



*Figure 15 — Raw Splunk event detail: Failure Reason "Unknown user name or bad password", Status 0xC000006D / Sub Status 0xC000006A, Workstation DESKTOP-T6T1N2V.*

## Analysis / Discussion

The detection logic is layered across both SIEM platforms to triangulate the same attack pattern:

- Rule escalation (Wazuh): rule 100100 fires on any single failed Windows logon (Event ID 4625); rule 100101 escalates once 5 matches occur within a 120-second window into one "Multiple Failed Logons - Possible Brute Force" alert tagged to MITRE ATT&CK T1110.
- Threshold search (Splunk): the SPL query groups 4625 events by source IP and flags 5 or more failed attempts, independently corroborating the Wazuh correlation threshold.
- Portability (Sigma): the single-event rule converted cleanly to OpenSearch/Wazuh syntax; the correlation rule did not, because the OpenSearch Lucene backend does not support Sigma correlation rules — portability via Sigma covers only the first-stage detection here.
- Confirmed detection-to-compromise chain: rule 100100 firing repeatedly, rule 100101 escalating at the threshold, and a successful NTLM logon (rule 92652) for the same account moments later — the textbook signature of a successful brute force.

The brute force succeeded because HR-STAFF1's password (Hope199872) was present in the test wordlist — a reminder that detection alone is not mitigation; the account should also be covered by a stronger password policy, MFA, and/or an automated lockout or active-response action once rule 100101 fires. The Sigma correlation-conversion failure is a useful finding in its own right: teams relying on Sigma for SIEM portability should verify correlation support in their target backend/pipeline before depending on it for multi-event detections.

## Conclusion

The lab objective was met and then some: a working brute-force detection pipeline was built, deployed, and validated against a live attack. CrackMapExec's dictionary attack against HR-STAFF1 succeeded (password Hope199872), and Wazuh detected it end to end — 118 authentication failures and 2 authentication successes were logged and correctly classified, with the custom correlation rule (100101: 5 failures / 120 seconds, MITRE T1110) escalating ahead of the successful compromise and a matching Successful Remote Logon alert (rule 92652) confirming it. The Splunk SPL query independently reproduced the same threshold logic, and the Sigma rule set made the first-stage detection portable across backends while also surfacing a concrete limitation worth documenting. Skills demonstrated include SIEM agent deployment, Sigma rule authoring and backend conversion, Wazuh custom/correlation rule writing, SPL query construction, and reading an attack's full signature — failed logons, escalation, and the eventual successful logon — straight out of the event log.

## Recommendations / Next Steps

- Enforce a stronger password policy (length/complexity, breached-password screening) and/or MFA for HR-STAFF1 and similarly privileged accounts — the dictionary attack succeeded well within the lab window.
- Add a Wazuh active-response action to automatically block or disable an account once rule 100101 fires, closing the gap between detection and compromise.
- Re-test the correlation rule against a Sigma backend/pipeline that supports correlation rules, or track the OpenSearch Lucene backend's roadmap for that feature.
- Expand the SOC Brute Force Detection Splunk dashboard beyond the single panel — add a time-series trend and a scheduled alert that fires when a source IP crosses the 5-failure threshold.
- Extend the pipeline to RDP and other remote-auth protocols using the existing Brute_Force_RDP.yml rule, which already converts and runs cleanly.

## Key Skills and Competencies Demonstrated

| Area | Demonstrated by |
|---|---|
| SIEM administration & correlation rule engineering | Deployed and tuned Wazuh rules 100100/100101 (threshold-based brute-force correlation, MITRE-tagged) |
| Threat detection content authoring (Sigma) | Authored vendor-neutral single-event and correlation Sigma rules; converted them to OpenSearch/Wazuh syntax |
| SPL development & dashboarding | Built a Splunk SPL query and the "SOC Brute Force Detection" dashboard |
| Windows/AD security event analysis | Interpreted raw Event ID 4625 logs to confirm the compromised account |
| Offensive tooling for defensive validation | Ran a controlled CrackMapExec dictionary attack to generate realistic telemetry |
| MITRE ATT&CK mapping | Tagged all detections to T1110 (Brute Force) |
| Incident investigation & reporting | Correlated Wazuh and Splunk evidence into a single root-cause narrative |
| Lab infrastructure & network design | Built and segmented a 4-VM environment (VirtualBox, NAT + host-only adapters, AD domain) |

## Reproducibility Notes

**Versions confirmed from the live environment:** Splunk 9.4.2 (Search & Reporting app); Wazuh dashboard/indexer reachable at 192.168.56.101/.102 on the 4.x line; Sigma CLI with the splunk, wazuh, opensearch, and sysmon backend plugins; domain HopeLabs.Local.

Checkpoints to confirm a reproduction is on track:

| After step | You should see |
|---|---|
| 0.7–0.9 (SIEM install) | sudo systemctl status wazuh-manager / wazuh-indexer / splunk status all report active/running |
| C (agent enrollment) | Test-NetConnection 192.168.56.102 -port 1514 and -port 1515 both return TcpTestSucceeded: True |
| E (attack run) | CrackMapExec returns STATUS_LOGON_FAILURE (not STATUS_NO_LOGON_SERVERS) once it reaches a real domain account, then [+] HopeLabs.Local\HR-STAFF1:<password> on a hit |
| G (Wazuh rule validation) | sudo /var/ossec/bin/wazuh-analysisd -t prints no errors |
| H / Results | The Wazuh Threat Hunting dashboard shows rule IDs 100100 and 100101 firing, and the Splunk search index=* EventCode=4625 returns matching events |

If a checkpoint doesn't match, the most common causes are: the host-only adapter not matching 192.168.56.0/24 on every VM, the Windows firewall blocking ports 1514/1515 before the Wazuh agent is enrolled, or the domain account used for the attack not existing yet in step 0.4.

## Full Command Reference (Step-by-Step Replication Log)

This is the exact run log, numbered as it was executed (Steps 1-34). Follow it in order on an environment built per Prerequisites and Base Infrastructure Setup and it reproduces the same detections. Narrative context for each step is in Methodology above; this section is the copy-paste reference.

**Step 1 — Build the wordlists** *(Kali)*.

```
nano usernames.txt      # 1,000+ entries
nano passwords.txt      # 1,000+ entries
```

**Step 2 — Install CrackMapExec and confirm connectivity** *(Kali)*.

```
sudo apt update
sudo apt install crackmapexec -y
crackmapexec --version
ping 192.168.56.116
```

**Step 3 — Check the Ubuntu IP** *(VM1)*.

```
ip addr          # -> 192.168.56.102, host-only interface
```

**Step 4 — Check the OS version** *(VM1)*.

```
lsb_release -a
```

**Step 5 — Check Wazuh services** *(VM1)*.

```
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer
```

**Step 6 — Check Splunk** *(VM1)*.

```
sudo systemctl status splunk
ls /opt/splunk/bin/splunk     # if the service name isn't found
```

**Step 7 — Verify connectivity to the other VMs** *(VM1)*.

```
ping -c 4 192.168.56.115      # Windows Server
ping -c 4 192.168.56.116      # Windows 10 client
```

**Step 8 — Start Wazuh and Splunk** *(VM1)*.

```
sudo systemctl start wazuh-manager
sudo systemctl status wazuh-manager
sudo /opt/splunk/bin/splunk start
sudo /opt/splunk/bin/splunk status
```

**Step 9 — Open Splunk in the browser** *(VM1)*.

```
http://192.168.56.102:8000
```

**Step 10 — Open the Wazuh dashboard**.

```
https://192.168.56.102:443
```

**Step 11 — Open PowerShell as Administrator** *(Windows 10/VM2)*. Start -> search "PowerShell" -> right-click -> Run as Administrator.

**Step 12 — Check the Windows IP** *(VM2)*.

```
ipconfig
```

**Step 13 — Test connectivity to Wazuh** *(VM2)*.

```
ping 192.168.56.102
```

**Step 14 — Check the agent communication port** *(VM2)*.

```
Test-NetConnection 192.168.56.102 -port 1514
```

**Step 15 — Check the enrollment port** *(VM2)*.

```
Test-NetConnection 192.168.56.102 -port 1515
```

Both return TcpTestSucceeded: True when reachable.

**Step 16 — Check/start the Wazuh agent service** *(VM2)*.

```
Get-Service "WazuhSvc"
Start-Service WazuhSvc
```

**Step 17 — Create the Sigma project structure** *(Kali)*.

```
mkdir -p Sigma_project/rules
```

**Step 18 — Install Sigma prerequisites** *(Kali)*.

```
sudo apt install git python3 python3-venv pipx -y
```

**Step 19 — Clone the official Sigma rules** *(Kali)*.

```
git clone https://github.com/SigmaHQ/sigma.git
```

**Step 20 — Confirm the files are present** *(Kali)*.

```
ls
```

**Step 21 — Install the Sigma CLI** *(Kali)*.

```
pipx ensurepath
pipx install sigma-cli
sigma --version
```

**Step 22 — Install the Sigma backend plugins** *(Kali)*.

```
sigma plugin install splunk
sigma plugin install wazuh
sigma plugin install opensearch
sigma plugin install sysmon
```

**Step 23 — Verify the Sigma install** *(Kali)*.

```
sigma plugin list
sigma list targets
sigma list pipelines
sigma list formats opensearch
```

**Step 24 — (Optional) clone the official rules to study** *(Kali)*.

```
git clone https://github.com/SigmaHQ/sigma.git ~/sigma-official
```

Note: cloning into the same directory as the working rules overwrites any pre-installed Sigma rules - clone to a separate path as shown.

**Step 25 — Run the brute-force attack against HR-STAFF1** *(Kali)*.

```
crackmapexec smb 192.168.56.116 -u usernames.txt -p passwords.txt -d HopeLabs.Local
```

**Step 26 — Write the Sigma rule** *(Kali)*.

```
cd Sigma_project/rules
nano Brute_Force_SMB.yml

title: Windows Failed Logon
id: 7c1f2a4e-3b8d-4e6a-9f10-2d5b8c3a1e01
name: win_failed_logon
status: experimental
description: Single failed Windows logon (Event ID 4625)
author: hope
date: 2026-10-05
```

```
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4625
  condition: selection
level: low
tags:
  - attack.credential_access
  - attack.t1110
---
title: Multiple Windows Failed Logons - Possible Brute Force
id: 9a3d5b7c-1e2f-4a6b-8c4d-5e7f9a1b2c02
status: experimental
description: 5 or more failed Windows logons for the same account within 2 minutes
author: hope
date: 2026-10-05
correlation:
  type: event_count
  rules:
    - win_failed_logon
  group-by:
    - TargetUserName
  timespan: 2m
  condition:
    gte: 5
level: high
tags:
  - attack.credential_access
  - attack.t1110
```

**Step 27 — List the OpenSearch Lucene pipelines** *(Kali)*.

```
sigma list pipelines opensearch_lucene
```

Returns empty by default — see Step 28.

**Step 28 — Create the custom pipeline and convert the rule** *(Kali)*.

```
nano wazuh_pipeline.yml

name: wazuh-windows-fields
priority: 20
transformations:
  - id: field_map
    type: field_name_mapping

sigma convert -t opensearch_lucene -p wazuh_pipeline.yml Brute_Force_SMB.yml
```

**Step 29 — Build the Splunk query** *(Splunk Search & Reporting)*.

```
index=* EventCode=4625
| rex "(?i)Source Network Address.*?(?<src_ip>\\S+)"
| stats count as failed_attempts by src_ip
| where failed_attempts >= 5
| sort -failed_attempts

# Alternative (counts only, no threshold filter):
index=* EventCode=4625
| stats count
```

**Step 30 — Build the Wazuh query** *(Wazuh Discover)*.

```
data.win.System.EventID: "4625"
```

**Step 31 — Write the Wazuh detection rules** *(VM1)*.

```
sudo nano /var/ossec/etc/rules/local_rules.xml

<rule id="100100" level="10">
  <if_sid>60122</if_sid>
  <description>Windows Failed Logon Detected</description>
  <group>authentication_failed</group>
</rule>

<!-- 5 failed windows logon (Event 4625) -->
<rule id="100101" level="12" frequency="5" timeframe="120">
  <if_matched_sid>100100</if_matched_sid>
  <description>Multiple Failed Logons - Possible Brute Force </description>
  <mitre>
    <id>T1110</id>
  </mitre>
  <group>authentication_failures,brute_force</group>
</rule>
```

**Step 32 — Validate the new rule** *(VM1)*.

```
sudo /var/ossec/bin/wazuh-analysisd -t
```

A successful run prints no errors.

**Step 33 — Confirm the alert in the Wazuh dashboard.** Wazuh dashboard → Threat Hunting → Events — filter or scroll to “Multiple Failed Logons - Possible Brute Force”.

**Step 34 — Confirm the alert in Splunk.** Splunk → Search & Reporting → Search:

```
index=* EventCode=4625
```

or, to reproduce the thresholded view from Step 29:

```
index=* EventCode=4625
| rex "(?i)Source Network Address.*?(?<src_ip>\\S+)"
| stats count as failed_attempts by src_ip
| where failed_attempts >= 5
| sort -failed_attempts
```

## References / Appendices

- Wazuh documentation: https://documentation.wazuh.com
- Splunk documentation: https://docs.splunk.com
- Sigma rule repository (SigmaHQ): https://github.com/SigmaHQ/sigma
- MITRE ATT&CK, Technique T1110 (Brute Force): https://attack.mitre.org/techniques/T1110/
- SecLists (username/password wordlists): https://github.com/danielmiessler/SecLists
- CrackMapExec: used for the SMB brute-force simulation against the HR-STAFF1 account

Full Sigma rule, Wazuh rule XML, and Splunk SPL query are included in full in the Methodology section above. All screenshots (Figures 1–15) were captured directly from the lab environment across the 2026-10-05, 2026-10-07, and 2026-10-14 sessions.